

University Number: _____

THE UNIVERSITY OF HONG KONG

**SCHOOL OF COMPUTING AND DATA SCIENCE
DEPARTMENT OF COMPUTER SCIENCE**

COMP7104/DASC7104 - Advanced Database Systems

Date: May 21, 2025

Time: 6:30pm-8:30pm

<INSTRUCTION ABOUT NUMBER OF QUESTION TO BE ANSWERED: OVERALL
AND FROM EACH SECTION>

23 questions overall, divided into two sections as follows:

Section 1 – Multiple-choice questions – 15 questions

Section 2 – Detailed-answer questions – 8 questions

<INDICATION OF COMPULSORY QUESTIONS>

None.

<INFORMATION ABOUT THE MARK VALUE OF QUESTIONS>

Section 1 – 2 points per question – 30 / 100 points in total

Section 2 – number of points depends on the question – 70 / 100 points in total

Only approved calculators as announced by the Examinations Secretary can be used in this examination. It is candidates' responsibility to ensure that their calculator operates satisfactorily, and candidates must record the name and type of the calculator used on the front page of the examination script.

Candidates are permitted to bring one sheet of A4-sized paper with printed or written notes on both sides to the examination.

Mark your University Number on each page (top right corner).

University Number: _____

Brand and Type of Calculator: _____

General indications

- For questions involving arithmetic expressions, numerical work or derivations, credit will be given only if the work leading to the answer is shown.

Section 1 – Multiple-choice questions – 15 questions – 2 points per question

Circle the true answer(s) to each question. Each question may have zero, one, or several valid answers. For each question, only the identification of all and only the valid answers will allow you to obtain the assigned points.

Q1.1 Which of the following assertions about join algorithms are true?

(A) Block (a.k.a. Chunk) Nested Loops Join will always perform at least as well as Simple (Page) Nested Loops Join in terms of I/O cost.

(B) For Nested Loop Join, to minimize I/O cost, it is best to make the smaller relation the outer branch of the loop.

(C) If we are joining two tables that are very different in size, using Index Nested Loop Join, and both relations have indexes on their join column, to minimize I/O cost, it is best to query the index of the smaller relation.

Explanation: A is true, since BNLJ turns into PNLJ when $B=2$. Otherwise the cost model is strictly less: $[R] + [R] \times [S]$ vs $[R] + [R/(B - 2)] \times [S]$. B is true: consider the cost model for PNLJ, where O is outer, I is inner. $[O] + [O] \times [I]$. C is false: consider the cost model for INLJ, where L is larger and S is smaller. If we query S, the cost is $[L] + \text{size}(L) \times (\text{cost to lookup in S})$, whereas if we query L the cost is $[S] + \text{size}(S) \times (\text{cost to lookup in L})$. The cost to lookup in an index will not change much even if one table is much bigger but the number of records can change drastically, so you want to lookup in the larger table.

Q1.2 Suppose we do a Simple (Page) Nested Loop Join on two tables, P and Q. The same amount of pages are required to store P and Q, but Q has three times more records per page than P, on average.

Which one of the following options is true ?

(A) $P \bowtie Q$ does fewer I/O operations than $Q \bowtie P$.

(B) $Q \bowtie P$ does fewer I/O operations than $P \bowtie Q$.

(C) $P \bowtie Q$ does the same number of I/O operations as $Q \bowtie P$.

Explanation: cost is $[P] \times [Q] + [P]$ or $[P] \times [Q] + [Q]$ so C.

Q1.3 Which of the following assertions about join algorithms are true?

(A) If both tables in a simple nested loop join fit entirely in main-memory, the order of inner and outer tables does not significantly affect I/O cost.

(B) If neither table fits entirely in main-memory, I/O cost would be lower if we process both tables on a per-block basis rather than per-tuple basis.

(C) If the smaller table fits in main-memory, while the largest one does not fit, we should use a hash-based join instead of a sort based one.

Explanation: A is true, if both tables fit entirely in memory, then they can be read just once and therefore the order would not be very important. B true: a block nested loop join has fewer disk accesses when compared to a simple nested loop join. C true: We can use Naive Hash Join.

Q1.4 A nested loop join will output a sorted stream of tuples under which of the following conditions?

(A) All nested loop joins will output a sorted stream.

(B) Outer table is already sorted.

(C) Inner table is already sorted.

Explanation: A nested loop join sequentially iterates through the outer table to find matching tuples from the inner table. If the outer table is already sorted, the nested loop join will preserve its order.

Q1.5 Consider the following join algorithm. Given an equijoin on tables R and S (with $[R] < [S]$), but no indexes on the join columns initially, the algorithm will take the following steps in sequence (1) bulk-load an Alternative-1 B+-tree on R, (2) perform an Index Nested Loops Join on S JOIN R using the index, and (3) delete the index on R. Assuming B (buffer size) $< [R] < [S] < (B-1)^2$ mark which statements are true.

(A) This algorithm generates more I/Os than Grace Hash Join (without recursive partitioning).

(B) This algorithm generates more I/Os than a 2-pass Sort-Merge join.

(C) This algorithm generates more random I/Os than Block Nested Loops Join.

Explanation: all items are true, since we need to sort the data in R.

Q1.6 Which join operators among the following ones are not pipeline breakers in query execution plans?

(A) Index Nested Loop Join.

(B) Chunk Nested Loop Join.

(C) Grace-Hash Join.

Explanation: as seen in lectures, A and B are true, tuples from the outer branch can be joined one a time / one chunk at a time, by lookup in the index of the inner branch or by scanning the inner branch. Hashing requires seeing all the input before being able to produce output.

Q1.7 Which of the following assertions about the System R / Selinger algorithm are true?

- (A) The resulting plan may include a cartesian product.
- (B) The resulting plan is always a pipelined one.
- (C) It only keeps the cheapest sub-plans as it proceeds from one stage to the next.

Explanation: A is true, as the query itself may have cartesian product, B is false, may include operators which are pipeline breakers. C is false, it also keeps cheapest for each interesting order.

Q1.8 Which statements about the System R / Selinger algorithm for query optimization are true?

- (A) It always guarantees the generation of an optimal execution plan by exhaustively evaluating all possible plans to ensure the lowest cost for query execution.
- (B) Predicate (selection) and projection pushdown will always improve performance.
- (C) It has a search algorithm that builds plans in top-down manner.

Explanation: A is false, not all plans are considered and no such guarantees can be provided, since it works with cost estimates. B is false: e.g. if the predicate involves an expensive function call. C is false: plans are built in bottom-up manner.

Q1.9 Which statements about histograms for cost and size estimates in the System R query optimizer are true?

- (A) Equi-width histograms use the same range size for each bucket.
- (B) Equi-depth histograms vary the range size of buckets so that the total number of occurrences for each bucket is roughly the same.
- (C) Unless we work with the assumption of uniform distribution within buckets, we can only use histograms for equality or range conditions that fall exactly on range boundaries.

Explanation: A and B are true by their definition, C is true, unless we have uniformity, and thus fraction of range is fraction of count.

Q1.10 Which statements about locking protocols and potential issues in concurrency control are true?

- (A) Cascading aborts are possible under Strict Two-Phase Locking (Strict 2PL).
- (B) 2PL guarantees a conflict-serializable schedule.
- (C) For a schedule following strong strict 2PL, the dependency graph is guaranteed to be acyclic.

Explanation: A is false: Strict 2PL prevents cascading aborts by holding all the locks until a transaction reaches its commit point, ensuring that other transactions do not see the intermediate, uncommitted data. B is true: 2PL guarantees a conflict-serializable schedule because it generates schedules whose precedence graph is acyclic. C is true: 2PL guarantees a conflict-serializable schedule and a schedule provided by Strict 2PL has even stronger guarantees

Q1.11 Which statements about concurrency control and concurrent schedules are true?

- (A) Dirty reads may occur in conflict-serializable schedules.
- (B) Lost updates may occur in conflict-serializable schedules.
- (C) Cascading aborts may occur in conflict serializable schedules.

Explanation: A and C are true: Conflict-serializability ensures that the schedule is conflict equivalent to a serial schedule. But dirty reads can still occur in conflict-serializable schedules, e.g. when we have cascading aborts. If a transaction reads data written by another transaction which later gets aborted, then the first transaction has read dirty data.

Q1.12 Which statements about parallel database architectures are true ?

- (A) In shared-disk architectures compute and storage can scale independently.
- (B) Shared-memory architectures are the closest to a centralized architecture.
- (C) In shared-nothing architectures, scaling the storage layer is easier than in shared-disk ones.

Explanation: A and B are true, as discussed in class. C is false, it is the opposite, since shared-nothing requires to move data around (eg to a new disk).

Q1.13 Which of the following assertions are true about data partitioning & query evaluation in PDBMSs?

- (A) If we partition data using the round robin scheme, insertion by key will incur a higher cost than with range or hash partitioning.
- (B) A join over two tables can benefit more from a round-robin partitioning of their records than from a hash partitioning on the join attribute.
- (C) A join over two tables can benefit more from a range partitioning on the join attribute than from a round-robin partitioning of their records.

Explanation: A is true since we need to check for duplicates at all nodes if round-robin. B is false (is the other way around), C is true since we can join independently in each range (at each node).

Q1.14 Which of the following are reasons to replicate data in NoSQL databases ?

- (A) To allow multiple nodes to split up the work for a large volume of requests.
- (B) To reduce latency by allowing clients to fetch data from a nearby server.
- (C). To handle ever-increasing database sizes.

Explanation: A is workload scalability, B is latency, C is data scalability so false.

Q1.15 Which of the following assertions are true for NoSQL database systems ?

- (A) They were designed to cope with high rates of simple read / write operations.

(B) They rely on replication for data scalability.

(C) They were designed for transactional workloads.

Explanation: A is true, high rate of simple read/write, which is also why C is true. B is false, it is for workload scalability.

Section 2 – Detailed-answer questions – 8 questions – 70 points in total

Q2.1 (2 points) – Joins - Assume we want to perform a join between two tables R and S using hashing. Assume that none of the tables is small enough so that its hash table (denoted HT_R and HT_S respectively) would fit in main memory, and that a large number of distinct values of the join attribute hash to the same bucket using a first hash function h_p . Would creating hash tables for R and S using h_p and then rehashing into another hash table using some hash function $h \neq h_p$ for large h_p buckets work? Explain why or why not?

Explanation: Yes, that is precisely Grace-Hash Join.

Q2.2 (4 points) Consider two relations R and S to be joined on their common attribute 'a' by $R.a = S.a$

- R has 40,000 and each record is 10 bytes long.
- S has 500 tuples, and each record is 50 bytes long.
- Page size is 1KB.
- The buffer pool is 50 pages large.
- The fill factor for all hash tables is 0.7.
- There are no indexes.

Considering all the join algorithms covered in this class, what is the minimum estimated I/O cost to execute this join? Exclude the final write I/O cost (saving the join result) from your solution.

Solution: 425

Explanation:

$[Product] = 40000 \times 10 / 1000 = 400$ pages

$[Company] = 500 \times 50 / 1000 = 25$ pages

We can perform a naive in-memory hash join, since $[Company] = 25 < \text{fill factor} \times (B - 2) = 0.7 \times (50 - 2) = 34$ (ceiling value)

The I/O cost is $[Product] + [Company] = 425$

Q2.3 (5 points) - Cardinality estimation - Consider a table Team(Id, name, city, country). You need to estimate the cardinality of the following query:

SELECT * FROM Team WHERE country= 'UK' AND city = 'London'

For the following questions, assume Team has 10,000 rows with 5% having country 'UK', and 0.3% having city = 'London'.

(a) (2.5 points) Under a uniform data assumption and independent predicates assumption, what is the estimated cardinality of this query?

Explanation: $[10\,000 \times 0.05 \times 0.003] = 1.5$

(b) (2.5 points) Is the result from previous question an overestimate or underestimate of the true cardinality?

Explanation: We have an underestimate of the true cardinalit, since country and city are not independent attributes - they are highly correlated, London is in the UK (or at least one London is in the UK). $\lceil 10\,000 \times 0.003 \rceil = 30$ if just one London city.

Q2.4 (11 points) - Serializability - Consider the schedule consisting of four transactions in the next table, where R and W stand for ‘Read’ and ‘Write’, respectively, and time flows from left to right.

Time	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈	t ₉	t ₁₀
T ₁					R(A)			R(B)	W(B)	
T ₂		R(E)	W(E)				R(D)	W(C)		W(A)
T ₃	R(B)		W(C)			W(A)				
T ₄			R(B)	R(D)	W(D)					

(a) (2 points) Is this schedule serial (explain) ?

Explanation: This schedule is not serial because this schedule interleaves the actions of different transactions. It is a concurrent schedule.

(b) (5 points) Compute the conflict dependency graph for this schedule.

Explanation: The answer is:

- T₁ → T₂(A), T₁ → T₃(A)
- T₃ → T₁(B), T₃ → T₂(A, C)
- T₄ → T₁(B), T₄ → T₂(D)

(c) (2 points) Is this schedule conflict-serializable (explain) ?

Explanation: This schedule is not conflict-serializable because there are cycles in its data dependency graph.

(d) (2 points) Is this schedule possible under regular (not strict) 2PL (explain)?

Explanation: This schedule is not possible under 2PL because it is not conflict serializable, and 2PL is guaranteed to produce conflict serializable schedules.

Q2.5 (12 points) – Deadlock avoidance - Consider the following lock requests of four transactions, where S stands for shared lock and X stands for exclusive lock. Assume that transactions never release a granted lock. Assume that in terms of priority $T_1 > T_2 > T_3 > T_4$ (e.g., T_1 is older than T_2 , etc).

(a) (6 points) Wait-Die. Complete the table, determining at each time step (time flows left to right) if the lock request is granted ('g'), blocked ('b'), aborted ('a'), or already dead ('-'), under the following policies.

time	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈
T ₁		X(A)						
T ₂				X(C)				
T ₃	X(A)		S(C)		X(B)			X(D)
T ₄						S(B)	X(C)	
LM	g							

Explanation: b g b g a - g

- t₂ blocked since T₃ already holds exclusive lock on A. T₁ is older than T₃ so it waits.
- t₃ granted, t₄ blocked since T₃ already holds shared lock on C. T₂ is older than T₃ so it waits.
- t₅ granted, t₆ aborted since T₃ already holds exclusive lock on B. T₄ is younger than T₃ so it aborts.
- t₇ transaction T₄ is already dead since aborted at t₆.
- t₈ granted

(b) (6 points) Wound-Wait. Complete the table, determining at each time step if the lock request is granted ('g'), blocked ('b'), killed (granted by aborting another transaction) ('k'), or the requester is already dead('−').

time	t ₁	t ₂	t ₃	t ₄	t ₅	t ₆	t ₇	t ₈
T ₁		X(A)						
T ₂				X(C)				
T ₃	X(A)		S(C)		X(B)			X(D)
T ₄						S(B)	X(C)	
LM	g							

Explanation: k – g – g b -

At t₂ killed since T₃ already holds exclusive lock on A. T₁ is older than T₃ so it causes T₃ to abort. At t₃ already dead, at t₄ granted, at t₅ already dead, at t₆ granted, at t₇ blocked since T₂ already holds exclusive lock on C. T₄ is younger than T₂ so it waits, at t₈ already dead.

Q2.6 (5 points) – Concurrency control – Consider the following schedule, involving two transactions, where times flows from left to right.

T₁: W(X) W(Y)
T₂: R(X) R(Y)

(a) (2.5 points) Is this schedule conflict serializable (explain) ?

(b) (2.5 points) Is this possible under regular (not strict) 2PL ?

Explanation: 2PL enforces conflict serializability but may not allow all conflict serializable schedules. This schedule is conflict-serializable according to the definition, but cannot be obtained by 2PL.

Q2.7 (18 points) - Selectivity Estimation - Consider a relation R(a, b, c) with 5000 tuples. We have an index on 'a' with 50 unique values in the range [1, 50] and an index on 'b' with 100 unique values in the range [1, 100]. We do not have an index on 'c'.

Use selectivity estimation to estimate the number of tuples produced by the following queries.

(a) (1.5 points) SELECT * FROM R

Explanation: 5000 (no predicates, so select all tuples)

(b) (1.5 points) SELECT * FROM R WHERE a=40

Explanation : Sel = 1 / unique values of a = $1/50$ 5000 * (1/50) = 100

(c) (1.5 points) SELECT * FROM R WHERE b=40

Explanation : Sel = 1 / unique values of b = $1/100$. 5000 * (1/100) = 50

(d) (1.5 points) SELECT * FROM R WHERE c=40

Explanation: Sel = 1/10 because we do not have any information on 'c' we use 1/10 as default.

$$5000 * (1/10) = 500$$

(e) (1.5 points) SELECT * FROM R WHERE a <= 10

Explanation: Sel = $(v - \text{low}) / (\text{high} - \text{low} + 1) + 1 / \text{distinct values} = (10 - 1) / (50 - 1 + 1) + 1 / 50 = 1/5$

$$5000 * (1/5) = 1000$$

(f) (1.5 points) SELECT * FROM R WHERE b <= 20

Explanation: Same formula as previous, or just look and see that the first 25 values are 1/5 of the values between 1 and 100: Sel = 1/5

$$5000 * (1/5) = 1000$$

(g) (1.5 points) SELECT * FROM R WHERE c <= 25

Explanation : Sel = 1/10

$$5000 * (1/10) = 500$$

(h) (1.5 points) SELECT * FROM R WHERE a <= 10 AND b <= 20

Explanation : Sel = Sel(a <= 10) * Sel(b <= 20) = 1/5 * 1/5 = 1/25 5000 * (1/25) = 200

(i) (1.5 points) SELECT * FROM R WHERE $a \leq 10$ AND $c \leq 25$

Explanation : $\text{Sel} = \text{Sel}(a \leq 10) * \text{Sel}(c \leq 20) = 1/5 * 1/10 = 1/50$ $5000 * (1/50) = 100$

(j) (1.5 points) SELECT * FROM R WHERE $a \leq 10$ OR $b \leq 20$

Explanation: $\text{Sel} = \text{Sel}(a \leq 10) + \text{Sel}(b \leq 20) - \text{Sel}(a \leq 10) * \text{Sel}(b \leq 20) = 1/5 + 1/5 - 1/5 * 1/5 = 9/25$ $5000 * (9/25) = 1800$

(k) (1.5 points) SELECT * FROM R WHERE $a=b$

Explanation: $\text{Sel} = 1/\max(\text{distinct values of } a, \text{distinct values of } b) = 1/\max(50, 100) = 1/100$ $5000 * (1/100) = 50$

(l) (1.5 points) SELECT * FROM R WHERE $a=c$

Explanation: We do not have information on c, so we just use the number of distinct values in a: $\text{Sel} = 1/50$ $5000 * (1/50) = 100$

Q2.8 (13 points) PDBMS - Distributed Query Evaluation – Consider the following schema:

- Player(Player_Id, name) – Primary Key Player_Id – record size 6=2+4 bytes

- PlayerCoach(Player_Id, Coach_Id, Nb_years) – Primary Key (Player_Id, Coach_Id) – record size $6=2+2+2$ bytes
- Coach(Coach_Id, name, Division_Id) – Primary Key Coach_Id – record size $8=2+4+2$ bytes
- Division(Division_Id, name) – Primary Key Division_Id – record size $6=2+4$ bytes

For performance reasons, these tables may be partitioned and / or replicated across nodes, such that all nodes can do a similar amount of work in parallel.

We anticipate four-way join queries like the following:

```
SELECT *
FROM Player P JOIN PlayerCoach PC ON P.Player_Id = PC.Player_Id
      JOIN Coach C ON PC.Coach_Id = C.Coach_Id
      JOIN Division D ON D.Division_Id = C.Division_Id
WHERE P.name LIKE 'A%'
```

for any 'A' initial letter.

- Pages are 4KB in size.
- There are 4 nodes in the system.
- The Player table contains 40,000 records.
- The PlayerCoach table contains 100,000 records.
- The Coach table contains 100 records.
- The Division table contains 25 records.

(a) (6 points) Assuming that initially all the data is located at a single node (any of the four nodes), which data partitioning and / or replication strategy minimizes the *total network data transfer* for the given query, including the initial costs for replication and or partitioning? Assuming final join results do not need to be collected at one central node, give the cost of this strategy in terms of number of pages transferred across the network.

--

Explanation: Replicate the smaller tables, Coach and Division, across all nodes, with cost 2 pages x 3 nodes to be transferred, so 6 pages. Partition by Player_Id both Player and PlayerCoach, so cost is 3 x $\text{ceil}((60+150)/4)$ transferred pages, so 159 pages. Joins can then be computed locally, so no need to exchange data further. So total cost is 159+6=165 pages to be initially transferred.

(b) (4 points) Assuming now the selected strategy from (a) is followed and that the usual reduction factor of the LIKE condition is 1/25, what is the estimated amount of data (join results) to be transferred back to a central node.

Explanation: locally, at each of the four nodes, the maximal join result size is 25000 records of 20 bytes, then the selection reduces that to 1250 records of 20 bytes, so 7 pages. 7 x 3 pages = 21 pages are to be exchanged.

(c) (3 points) Assume now the following query must be computed instead, involving grouping and Max aggregation, with each of the four nodes computing as much as possible locally, including a partial aggregation, before sending their result to one central node among them for the final aggregation. What is the amount of data transferred over the network in this case?

```
SELECT D.name, Max(PC.Nb_Years)
FROM Player P JOIN PlayerCoach PC ON P.Player_Id = PC.Player_Id
      JOIN Coach C ON PC.Coach_Id = C.Coach_Id
      JOIN Division D ON D.Division_Id = C.Division_Id
WHERE P.name LIKE 'A%'
GROUP BY D.name
```

University Number: _____

Explanation: Since we have 4 nodes and 25 records in Division, this translates into an upper bound of 25 records per node (1 page); so 3 pages only are transferred.

END OF PAPER